# 🗓️ Week 1: Identity Foundations & Metabolic Calculators

## Sprint 1 : Core Identity Registration & Login Gateways

- **User Story:** As an unauthenticated client, I want to execute basic registration, profile initiation, and credential validation requests so that my system security boundaries are established.
- **Acceptance Criteria:**
  - `POST /register` must validate that the email format is unique and enforce strong credentials (minimum 6 characters, one uppercase, one number).
  - `POST /login` must log entries inside `LoginAttempts`. If failures hit 5 within 15 minutes, flip `IsLockedOut = true`, compute `LockedUntil` (+15 minutes), and throw `AUTH_ACCOUNT_LOCKED`.

### Endpoints Included & Specified

- `POST /api/v1/auth/register`

JSON

```
// Request Payload
{
  "firstName": "Ahmed",
  "lastName": "Mohamed",
  "email": "ahmed@example.com",
  "password": "Ahmed@123",
  "phoneNumber": "+201234567890"
}
// Response (201 Created)
{
  "isSuccess": true,
  "message": "Registration successful. Please complete your profile.",
  "data": { "userId": 1, "requiresProfileCompletion": true },
  "errors": [],
  "statusCode": 201,
  "timestamp": "2026-06-17T17:00:00Z"
}
```

- `POST /api/v1/auth/complete-profile`
  - *Payload:* `{}` → *Response (200):* `{"isSuccess": true, "message": "Profile lifecycle initiated.", "data": null, "errors": [], "statusCode": 200,`

`"timestamp": "2026-06-17T17:01:00Z"}`

- `POST /api/v1/auth/login`
  - *Payload:* `{"email": "ahmed@example.com", "password": "Ahmed@123"}`
  - *Response (200):* `{"isSuccess": true, "message": "Login successful", "data": { "token": "JWT_STR", "refreshToken": "RF_STR", "profileCompleted": true, "isPremium": false }, "errors": [], "statusCode": 200, "timestamp": "2026-06-17T17:02:00Z"}`

### Database Impacts & Error Mappings

- **Tables:** `Users` (Insert/Update), `LoginAttempts` (Insert).
- **Errors:** `AUTH_EMAIL_EXISTS (409)`, `AUTH_INVALID_CREDENTIALS (401)`, `AUTH_ACCOUNT_LOCKED (423)`.

## Sprint 2 : Security Token Management & Session Revocation

- **User Story:** As an authenticated user, I want to use my long-lived token string or terminate my active validation context so that my token states rotate safely.
- **Acceptance Criteria:**
  - Exchanging tokens via `/refresh-token` must look up the existing token inside `RefreshTokens`, check if it has been revoked or has expired, invalidate it, and insert a new token pair.
  - `/logout` must instantly terminate the current token claim context.

### Endpoints Included & Specified

- `POST /api/v1/auth/refresh-token`

JSON

```
// Request Payload
{ "refreshToken": "refresh_token_here" }
// Response (200 OK)
{
  "isSuccess": true,
  "message": "Token refreshed successfully",
  "data": { "token": "new_jwt_string", "refreshToken": "new_refresh_string" },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:05:00Z"
}
```

- `POST /api/v1/auth/logout`

- *Payload:* None → *Response (200):* `{"isSuccess": true, "message": "Logged out successfully", "data": null, "errors": [], "statusCode": 200, "timestamp": "2026-06-17T17:06:00Z"}`

### Database Impacts & Error Mappings

- **Tables:** `RefreshTokens` (Update/Insert).
- **Errors:** `AUTH_TOKEN_EXPIRED (401)`, `AUTH_TOKEN_INVALID (401)`.

## Sprint 3 : Password Recovery & OTP Handshakes

- **User Story:** As an unauthenticated user locked out of my account, I want to utilize an email OTP workflow so that I can securely reset my credentials.
- **Acceptance Criteria:**
  - Generating an OTP code must store its hash inside `OtpCodes` with an expiration window of exactly 10 minutes.
  - Limit code re-generation to once every 30 seconds. `POST /reset-password` requires verification matching against the generated `resetToken`.

### Endpoints Included & Specified

- `POST /api/v1/auth/forgot-password`
  - *Payload:* `{"email": "ahmed@example.com"}` → *Response (200):* `{"isSuccess": true, "message": "Verification code sent", "data": {"otpExpiresIn": 600, "canResendIn": 30}, "errors": [], "statusCode": 200, "timestamp": "2026-06-17T17:10:00Z"}`
- `POST /api/v1/auth/verify-otp`

JSON

```
// Request Payload
{ "email": "ahmed@example.com", "otp": "123456" }
// Response (200 OK)
{
  "isSuccess": true,
  "message": "OTP verified successfully",
  "data": { "resetToken": "reset_token_xyz" },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:11:00Z"
}
```

- `POST /api/v1/auth/reset-password`

- *Payload:* `{"resetToken": "xyz", "newPassword": "NewPass@123", "confirmPassword": "NewPass@123"}` → *Response (200):* `{"isSuccess": true, "message": "Password reset successfully", "data": {"passwordChanged": true}, "errors": [], "statusCode": 200, "timestamp": "2026-06-17T17:12:00Z"}`

### Database Impacts & Error Mappings

- **Tables:** `OtpCodes` (Insert/Update), `Users` (Update PasswordHash).
- **Errors:** `AUTH_INVALID_OTP (400)`, `AUTH_OTP_EXPIRED (400)`, `AUTH_PASSWORD_MISMATCH (400)`.

## Sprint 4 : Demographic Metadata & Preferences Engine

- **User Story:** As an authenticated athlete, I want to customize my personal metrics, upload my avatar file, and modify my regional UI preferences so that my application profile settings are personalized.
- **Acceptance Criteria:**
  - Profile photo multi-part uploads must restrict binary file formats to JPG/PNG and enforce a maximum file size limit of 5MB.
  - Settings configuration adjustments must accept partial data patches instead of forcing full object data replacements.

### Endpoints Included & Specified

- `GET /api/v1/profile`
- `PUT /api/v1/profile`
- `POST /api/v1/profile/picture`
- `PUT /api/v1/profile/change-password`
- `GET /api/v1/settings`
- `PUT /api/v1/settings`

JSON

```
// Request Payload for PUT /api/v1/settings (Partial Update)
{
  "theme": "dark",
  "notifications": { "workoutReminders": true }
}
// Response (200 OK)
{
  "isSuccess": true,
  "message": "Settings updated safely.",
```

```
  "data": null,
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:20:00Z"
}
```

### Database Impacts & Error Mappings

- **Tables:** `UserProfiles`, `UserPreferences`, `NotificationSettings`, `PrivacySettings` (All row updates mapped via `UserId`).
- **Errors:** `VAL_REQUIRED_FIELD (400)`, `VAL_FILE_TOO_LARGE (400)`, `RES_NOT_FOUND (404)`.

## Sprint 5 : FCE Ingestion & Metabolic Calculations

- **User Story:** As an onboarding trainee, I want the system to process my raw biological measurements through standard equations so that my custom BMR, TDEE, and calorie allocations are derived.
- **Acceptance Criteria:**
  - Validate numeric boundaries: Age (16–100), Weight (40–200kg), Height (140–220cm).
  - Floating-point mathematical formulas must apply gender criteria dynamically:
    - **Male BMR:**

$$BMR = 10 \times \text{weight}_{\text{kg}} + 6.25 \times \text{height}_{\text{cm}} - 5 \times \text{age} + 5$$

    - **Female BMR:**

$$BMR = 10 \times \text{weight}_{\text{kg}} + 6.25 \times \text{height}_{\text{cm}} - 5 \times \text{age} - 161$$

  - Compute TDEE using the designated activity factor multipliers (Rookie=1.2 up to TrueBeast=1.9). Calculate final calorie targets by goal orientation (e.g., Lose Weight = $TDEE - 500$).

### Endpoints Included & Specified

- `POST /api/v1/fitness/weight-goal-activity`
- `POST /api/v1/fitness/calculate`

JSON

```
// Request Payload for POST /api/v1/fitness/calculate
{ "userId": "usr_123456" }
// Response (200 OK)
{
```

```
  "isSuccess": true,
  "message": "Fitness metrics calculated successfully",
  "data": {
    "bmr": 1728.75,
    "tdee": 2679.56,
    "calorieTarget": 2179.56,
    "status": "Normal"
  },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:30:00Z"
}
```

- `GET /api/v1/fitness/metrics/{userId}`
- `GET /api/v1/fitness/stats/{userId}`

### Database Impacts & Error Mappings

- **Tables:** `UserFitnessStats` (Insert), `CalculatedMetrics` (Upsert matched on `UserId`).
- **Errors:** `VAL_INVALID_WEIGHT (400)`, `VAL_INVALID_GENDER (400)`, `FCE_STATS_NOT_FOUND (404)`.

## Sprint 6 : FCE Automated Plan Mapping & Immutable History Logs

- **User Story:** As an athlete with recalculating biometrics, I want the system to assign an optimal fitness program matching my calorie status while retaining my past tracks inside an immutable audit log.
- **Acceptance Criteria:**
  - Query `FitnessPlanConfig` intersecting user `Goal` and calculation `Status` (`Weak` / `Normal` / `Hard`).
  - When reassignment rules execute, flip `IsActive = false` on the current active row, append a detailed tracking snapshot to `UserPlanHistory`, and create a new row in `UserAssignedPlans`.

### Endpoints Included & Specified

- `POST /api/v1/fitness/assign-plan`
- `PUT /api/v1/fitness/recalculate/{userId}`

JSON

```
// Request Payload
{ "reason": "weight_update", "newWeight": 73.0, "triggeredBy":
```

```
"progress_service" }
// Response (200 OK)
{
  "isSuccess": true,
  "message": "Metrics recalculated and plan history logged.",
  "data": { "previousStatus": "Normal", "newStatus": "Normal",
"planReassignment": true },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:35:00Z"
}
```

- `GET /api/v1/fitness/plan-configs`
- `GET /api/v1/fitness/plans/{planId}`

### Database Impacts & Error Mappings

- **Tables:** `UserAssignedPlans` (Update/Insert), `UserPlanHistory` (Insert).
- **Errors:** `FCE_NO_MATCHING_PLAN (404)`, `FCE_METRICS_NOT_CALCULATED (400)`.

## Sprint 7 : Workout Catalog Directory Queries

- **User Story:** As a training athlete, I want to query the master exercise library and browse workout program tracks using target criteria filters so that I can look up specific movement requirements.
- **Acceptance Criteria:**
  - Support index-optimized search parameters: `page`, `pageSize`, `category`, `difficulty`, `duration`, and free text string `search`.
  - Database row extraction must perform clean joins across target tables to map ordered lists sorted sequentially by `OrderIndex` ASC.

### Endpoints Included & Specified

- `GET /api/v1/workouts`

JSON

```
// Response Schema Example for GET /api/v1/workouts?
category=chest&difficulty=intermediate
{
  "isSuccess": true,
  "message": "Workouts fetched successfully.",
  "data": [
    { "workoutId": 12, "name": "Hypertrophy Chest A", "durationInMinutes": 45,
```

```
"difficulty": "Intermediate" }
  ],
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T17:40:00Z"
}
```

- `GET /api/v1/workouts/{id}`
- `GET /api/v1/workouts/by-plan/{planId}`
- `GET /api/v1/workouts/category/{categoryName}`

### Database Impacts & Error Mappings

- **Tables:** Read operations against `Workouts`, `WorkoutPlans`, and relational execution configurations.
- **Errors:** `RES_WORKOUT_NOT_FOUND (404)`, `RES_NOT_FOUND (404)`.

# 🗓️ Week 2: Content Libraries, Progress Tracking & Advanced Architecture

## Sprint 8 : Master Exercise Catalogs & Program Layouts

- **User Story:** As a developer building client components, I want to expose endpoints to isolate specific exercise identifiers and high-level training layouts so that full step instructions are viewable.
- **Acceptance Criteria:**
  - Detail retrievals for exercise items must return targeted labels, muscle grouping arrays, and video uniform resource identifiers ( `VideoUrl` ).

### Endpoints Included & Specified

- `GET /api/v1/exercises`
- `GET /api/v1/exercises/{id}`

JSON

```
// Response for GET /api/v1/exercises/45
{
  "isSuccess": true,
  "message": "Exercise resolved.",
  "data": {
    "exerciseId": 45,
    "name": "Barbell Bench Press",
```

```
    "targetMuscles": "Chest, Triceps",
    "equipment": "Barbell, Bench",
    "description": "Lower barbell smoothly to mid-chest levels and press up
dynamically.",
    "videoUrl": "/videos/exercises/bench.mp4"
  },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T18:00:00Z"
}
```

- `GET /api/v1/workout-plans`
- `GET /api/v1/workout-plans/{planId}`

### Database Impacts & Error Mappings

- **Tables:** Reads targeting `Exercises`, `WorkoutPlans`, and `WorkoutExercises` join indices.
- **Errors:** `RES_EXERCISE_NOT_FOUND (404)`, `RES_PLAN_NOT_FOUND (404)`.

## Sprint 9 : Workout Execution Session Lifecycle Ingestion

- **User Story:** As an athlete kicking off a physical workout, I want to instantiate an active runtime tracking session so that my actual volume, set-by-set metrics, and workloads are logged.
- **Acceptance Criteria:**
  - Starting a workout must add a record to `WorkoutSessions` marked as `"Active"`, returning a unique `SessionId` token.
  - Completing logs must parse a nested collection mapping execution metrics directly down to individual rows inside `WorkoutLogExercises`.
  - Close out the tracked row by transitioning `WorkoutSessions.Status` to `"Completed"`.

### Endpoints Included & Specified

- `POST /api/v1/workouts/{id}/start`
- `POST /api/v1/progress/workouts`

JSON

```
// Request Payload for logging performance
{
  "workoutId": 1,
  "sessionId": "sess_uuid_999",
  "completedAt": "2026-06-17T18:15:00Z",
```

```
  "duration": 45,
  "caloriesBurned": 350,
  "difficulty": "Intermediate",
  "notes": "Pushed to failure safely on the final sets",
  "rating": 5,
  "exercisesCompleted": [
    { "exerciseId": 12, "setsCompleted": 3, "repsCompleted": 12, "weightUsed":
80.0, "completed": true }
  ]
}
// Response (201 Created)
{
  "isSuccess": true,
  "message": "Workout completion logged successfully.",
  "data": { "logId": 4567, "streakUpdated": true, "currentStreak": 8 },
  "errors": [],
  "statusCode": 201,
  "timestamp": "2026-06-17T18:16:00Z"
}
```

### Database Impacts & Error Mappings

- **Tables:** `WorkoutSessions` (Update), `WorkoutLogs` (Insert), `WorkoutLogExercises` (Multi-row Insert), `Streaks` (Counter Update), `UserStatistics` (Update Accumulators).
- **Errors:** `RES_SESSION_NOT_FOUND (404)`, `VAL_REQUIRED_FIELD (400)`.

## Sprint 10 : Biometric Tracking Dashboards & Metrics Snapshots

- **User Story:** As an athlete reviewing my metrics history, I want to query my accumulated dashboards and log new weight snapshots so that my progress logs update and trigger background recalculations.
- **Acceptance Criteria:**
  - Inbound numeric inputs for tracking weight must validate within structural boundary limits (40–200 kg).
  - **Asynchronous Message Broker Hook:** Upon completing a successful database write to `WeightHistory`, the service must publish an internal background event (`weight_updated`) targeting the FCE to recalculate metabolic parameters automatically.

### Endpoints Included & Specified

- `GET /api/v1/progress`
- `GET /api/v1/progress/{userId}`

- `POST /api/v1/progress/weight`

JSON

```
// Request Payload
{ "weight": 72.5, "date": "2026-06-17", "notes": "Morning tracking" }
// Response (201 Created)
{
  "isSuccess": true,
  "message": "Weight entry updated. Async recalculation event dispatched.",
  "data": { "bmi": 23.7, "differenceFromPrevious": -0.5, "totalWeightLost":
3.0 },
  "errors": [],
  "statusCode": 201,
  "timestamp": "2026-06-17T18:30:00Z"
}
```

- `GET /api/v1/progress/weight-history/{userId}`
- `GET /api/v1/progress/achievements`
- `GET /api/v1/progress/stats/{userId}`

### Database Impacts & Error Mappings

- **Tables:** `WeightHistory` (Insert), `UserStatistics` (Update), `BodyMeasurements` (Read).
- **Errors:** `VAL_INVALID_WEIGHT (400)`, `RES_USER_NOT_FOUND (404)`.

## Sprint 11 : Nutrition Repositories & Calorie Allocation Matchers

- **User Story:** As an active user planning daily nutrition targets, I want to search recipes and discover suggested meal structures matched to my energy limits so that my eating plan fits my physical targets.
- **Acceptance Criteria:**
  - Personalized queries must execute a synchronous internal HTTP/REST callback down to the FCE to pull the user's active calculated `CalorieTarget`.
  - Match results against designated meal components type categories ( `Breakfast`, `Lunch`, `Dinner` ) where item caloric values fit the target range.

### Endpoints Included & Specified

- `GET /api/v1/nutrition/recommendations`
- `GET /api/v1/nutrition/recommendations/{userId}`

JSON

```
// Response for GET /api/v1/nutrition/recommendations/usr_123456
{
  "isSuccess": true,
  "message": "Personalized meal recommendations resolved.",
  "data": {
    "userDailyGoalCalories": 2179.56,
    "recommendedMeals": [
      { "mealId": 89, "name": "Grilled Chicken Lunch", "type": "Lunch",
"calories": 650, "protein": 55.0 }
    ]
  },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T18:45:00Z"
}
```

- `GET /api/v1/nutrition/meals/{id}`
- `GET /api/v1/nutrition/meal-plans`
- `GET /api/v1/nutrition/meal-plans/by-calories`

### Database Impacts & Error Mappings

- **Tables:** Reads targeting `Meals`, `MealPlans`, and `MealPlanItems` data maps.
- **Errors:** `RES_MEAL_NOT_FOUND (404)`, `FCE_METRICS_NOT_CALCULATED (400)`.

## Sprint 12 : Smart Coach Sessions & Feed Caching (LOW PRIORITY)

- **User Story:** As an active user, I want my coaching conversation blocks archived systematically and my home feed compiled through automated background caches so that my dashboards load instantly without hitting database bottlenecks.
- **Acceptance Criteria:**
  - Chat lines must log messages matching strictly to role structures: `user` or `assistant`.
  - `GET /home` must check `RecommendationCache` first ($CurrentTime < ExpiresAt$). If cache parameters hit an expired window, aggregate fresh payload records across sub-services, update the cache table, and output the data.

### Endpoints Included & Specified

- `POST /api/v1/smart-coach/chat`

JSON

```
// Request Payload
{ "message": "How do I optimize my target metrics?", "sessionId": "sess_111" }
// Response (200 OK)
{
  "isSuccess": true,
  "message": "AI reply generated.",
  "data": { "reply": "Based on your Lose Weight goal...",
"followUpSuggestions": ["Adjust protein ratios?"] },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T19:00:00Z"
}
```

- `GET /api/v1/smart-coach/history`
- `GET /api/v1/home`

### Database Impacts & Error Mappings

- **Tables:** `ChatSessions` (Insert), `ChatMessages` (Insert), `RecommendationCache` (Read/Upsert).
- **Errors:** `RES_SESSION_NOT_FOUND (404)`, `SRV_SERVICE_UNAVAILABLE (503)`.

## Sprint 13 : Subscription Billings & Account Tier Shifters (LOWEST PRIORITY)

- **User Story:** As an account user, I want to execute simulated plan transformations and toggle auto-renewal configurations so that my premium access boundaries are updated safely.
- **Acceptance Criteria:**
  - Upgrading plans must log entries to `BillingLogs`, write rows into `UserSubscriptions` modifying access markers ( `Tier = 'Premium'` ), and compute plan lifetimes.
  - **Asynchronous Integration Hook:** Post an event framework notice ( `subscription_upgraded` ) to clear out soft-capped gates across connecting systems automatically.

### Endpoints Included & Specified

- `GET /api/v1/subscription/status/{userId}`
- `POST /api/v1/subscription/upgrade`

JSON

```
// Request Payload
{ "userId": "usr_123456", "planTier": "Premium", "durationMonths": 1 }
// Response (200 OK)
{
  "isSuccess": true,
  "message": "Payment simulation passed. Account tier upgraded successfully.",
  "data": { "activeTier": "Premium", "expiresAt": "2026-07-17T19:15:00Z" },
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T19:15:00Z"
}
```

- `POST /api/v1/subscription/cancel`

### Database Impacts & Error Mappings

- **Tables:** `UserSubscriptions` (Insert/Update), `BillingLogs` (Insert Log Trace).
- **Errors:** `PERM_PREMIUM_REQUIRED (403)`, `SRV_DATABASE_ERROR (500)`.

## Sprint 14 : Notification Service & Gateway Middleware Stubs (LOWEST PRIORITY)

- **User Story:** As an active client, I want to fetch my unread in-app message alerts while the background worker logs broker integration signals securely at edge proxy layers.
- **Acceptance Criteria:**
  - The notification background thread worker must ingest integration events (e.g., `achievement_earned`) and add line items into `InAppNotifications` with `IsRead = false`.
  - **Gateway Mapping Validation:** Route parameters enforce structural hard limits on incoming traffic checks, filtering via the standard matrix rule bounds: Anonymous (50/hr), Free (500/hr), and Premium (2000/hr).

### Endpoints Included & Specified

- `GET /api/v1/notifications`

JSON

```
// Response Output Mapping for GET /api/v1/notifications
{
  "isSuccess": true,
  "message": "Notifications fetched successfully.",
  "data": [
```

```
    { "id": 78, "title": "7-  Streak!", "message": "Worked out 7 consecutive
s", "isRead": false }
  ],
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T19:30:00Z"
}
```

- `PUT /api/v1/notifications/{id}/read`

### Database Impacts & Error Mappings

- **Tables:** `InAppNotifications` (Read/Update matched via identity parameter indexes).
- **Errors:** `RATE_LIMIT_EXCEEDED (429)`, `RES_NOT_FOUND (404)`.

## 🏁 Technical Sprint Delivery Audit Matrix

| Daily Sprint Target | Domain Focus | Endpoints Included | Priority Level | Core Deliverable Focus |
|---|---|---|---|---|
| **Sprint 1** | Authentication Service | **3** | **Critical** | Sign-up pipelines, data hashing, password checks, lockout rules. |
| **Sprint 2** | Authentication Service | **2** | **Critical** | Token swap command routes and session termination tracking loops. |
| **Sprint 3** | Authentication Service | **3** | **Critical** | 6-digit challenge token hashes and verification reset lines. |
| **Sprint 4** | User Profile Service | **6** | **High** | Multi-part avatar handlers and preference configuration blocks. |
| **Sprint 5** | Fitness Calculation Engine | **4** | **High** | Biometric boundary checking and floating-point BMR/TDEE math. |
| **Sprint 6** | Fitness Calculation Engine | **4** | **High** | Config rule resolution steps and plan mutation audit trails. |
| **Sprint 7** | Workout Service | **4** | **High** | Sorted multi-table inner joins and paginated routine selectors. |

| Daily Sprint Target | Domain Focus | Endpoints Included | Priority Level | Core Deliverable Focus |
|---|---|---|---|---|
| **Sprint 8** | Workout Service | **4** | Medium | Muscle movement detail layouts and structural program libraries. |
| **Sprint 9** | Workout & Progress Logs | **2** | Medium | Runtime instance trackers and nested set performance ingestion. |
| **Sprint 10** | Progress Tracking Service | **6** | Medium | Weight logging snapshots and background event broadcast triggers. |
| **Sprint 11** | Nutrition Service | **5** | Medium | Inter-service target sync calls and calorie range matchers. |
| **Sprint 12** | Smart Coach Service | **3** | **Low** | Chat thread history structures and feed aggregator cache stubs. |
| **Sprint 13** | Subscription & Billing | **3** | **Lowest** | Receipt checkout auditing trails and upgraded tier events. |
| **Sprint 14** | Notification Service & GW | **2** | **Lowest** | Decoupled alert queue consumers and edge filter validation stubs. |
| **TOTALS** | **9 Microservices** | **51 APIs** | | **Complete backend coverage mapped sequentially.** |